

RISC V 32 bit Single cycle CPU Debugging - Prerna Agarwal

Problems:

- 1) The issue is that during reset, **Addr** (the PC) is being reset to 0, but **the fetch is happening combinatorially** — so **RAM[0]** is being read fine.
Machine code = 00500113 (fetched)

Intuition:

What I thought -> i should use a reset port in instruction memory so that when reset is asserted no machine code will be fetched.

But **no — you should NOT add reset to instruction memory.**

Because instruction memory is READ ONLY (ROM) , putting reset architecturally is not correct.

Root Cause :

- 1) The current Program Counter (PC) is not setting to reset correctly.

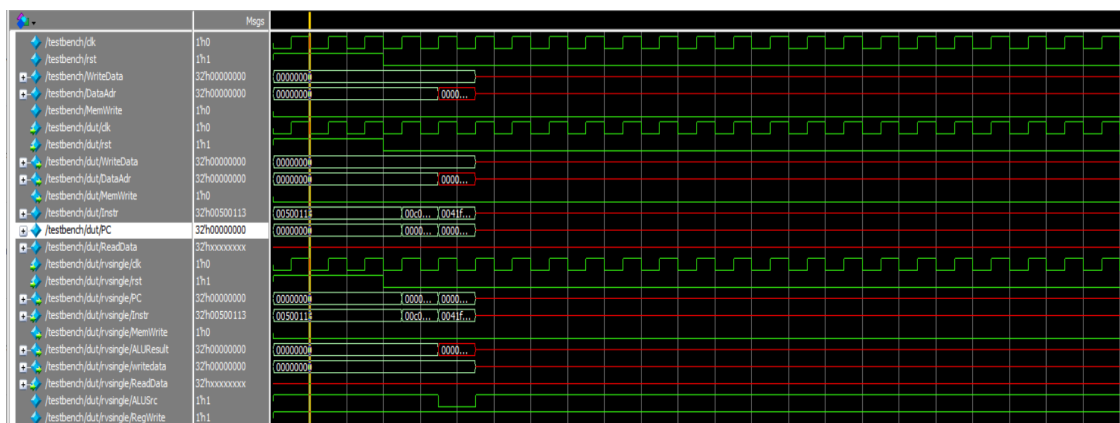
RST asserted

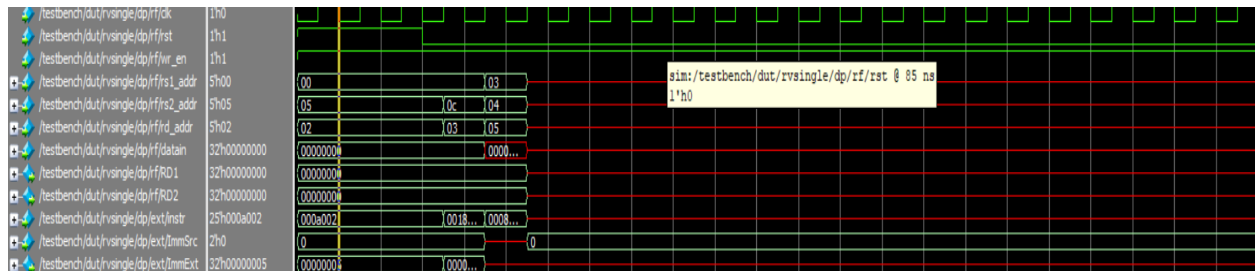
- PC = 0 ✓ correct
- Instr = RAM[0] ✓ correct
- Decode rs1, rs2 ✓ correct
- RegFile[rs1] = x ✗ PROBLEM — uninitialized (if no reset logic inside design)
- ALU input = x ✗ propagates
- Everything downstream = x ✗

- 2) In Register file , all registers = 0 on reset

Only **x0(zero register) = 0** is guaranteed

Other registers are **undefined after reset** unless software initializes them. Also it depends on the operation whether we are writing into a register file or not .





In my design, reset logic is present in the Register file and PC . so we can verify with the waveform provided.

Changes done:

In Testbench :

```
rst<=1;
```

```
#15; //reduce the delay
```

```
rst <= 0;
```

2) Before Jump to huge program testing which is going to be loaded in instruction memory

First execute simple R type , lw, sw type instruction to check correctness of datapath and control path.

#	Instruction	Machine code	Operation	Result	Memory effect
1	R-type or x4, x5, x6	0062E233	$x4 = x5 \mid x6$ $= 6 \mid 10$ $= 0110 \mid 1010$ $= 1110 = 14$	$x4 \leftarrow 14$ written on posedge clk via wr_en + rd_addr=4	none
2	S-type sw x6, 8(x9)	0064A423	Addr = x9 + 8 $= 4 + 8 = 12$ word idx = $12 \div 4 = 3$ WD = x6 = 10	no rd written	$RAM[3] \leftarrow 10$ byte addr 12 WE=1 on posedge clk
3	I-type lw x7, -4(x9)	FFC4A303	Addr = x9 + (-4) $= 4 - 4 = 0$ word idx = $0 \div 4$ $= 0$ RD = RAM[0]	$x7 \leftarrow 0$ RAM[0] was never written $\rightarrow$ reads 0	(read only)

Modified Instruction Memory , Data Memory , Register file

```

assign RD1 = (rs1_addr!= 0)? REG[rs1_addr] :0;
assign RD2 = (rs2_addr!= 0)? REG[rs2_addr] :0;

always @(posedge clk) begin
    if(rst)begin
        for(int i=0 ;i<32;i++)
            REG[i] <= 32'b0;
        end
    else if(!rst) begin
        REG[0] = 32'd0;
        REG[1]= 32'd3;
        REG[2] = 32'd2;
        REG[3] = 32'd9;
        //REG[4] = 32'd6;
        REG[5] = 32'd6;
        REG[6] = 32'd10;
        REG[7] = 32'd0;
        REG[8] = 32'd0;
        REG[9] = 32'd4;

        end
    else if( wr_en && rd_addr!=0)begin
        REG[rd_addr] <= datain;
    end
end
endmodule

```

```

module Datamem( input logic clk , WE ,rst,
                input logic [31:0] Addr ,WD ,
                output logic [31:0] RD
                );

    //memory
    logic [31:0] RAM[100:0];

    always_ff @(posedge clk) begin
        if(rst) begin
            // initial begin
                for(int i =0 ; i<=100 ; i++)
                    RAM[i] <= 32'd0;
            end
            //then set specific locations
            else if (!rst)
                RAM[0] = 32'd0; //byte address 0
                RAM[1] = 32'd2; //bytre addr 4
                RAM[2] = 32'd7; //byte addr 8
                RAM[3] = 32'd0; //byte addr 12

            end
        //write
        //always_ff @(posedge clk)

            else if(WE)
                RAM[Addr[31:2]] <= WD;

        end

        //read
        assign RD = RAM[Addr[31:2]];

    endmodule

```

```

module Instrmem( input logic [31:0] Addr , output logic [31:0] machine_code);

//memory
logic [31:0] RAM[63:0] ; //locations could be larger but taking here 64

// initial
//
// $readmemh("riscvtest.txt", RAM); //load hex file

initial begin
    RAM[0] = 32'h0062E233;
    RAM[1] = 32'h0062E233;
    RAM[2] = 32'h00644A03;
    RAM[3] = 32'hFFC4A303;
end

assign machine_code = RAM[Addr[31:2]]; //why 0 and 1 bit left see micro

endmodule

```

Results Obtained:

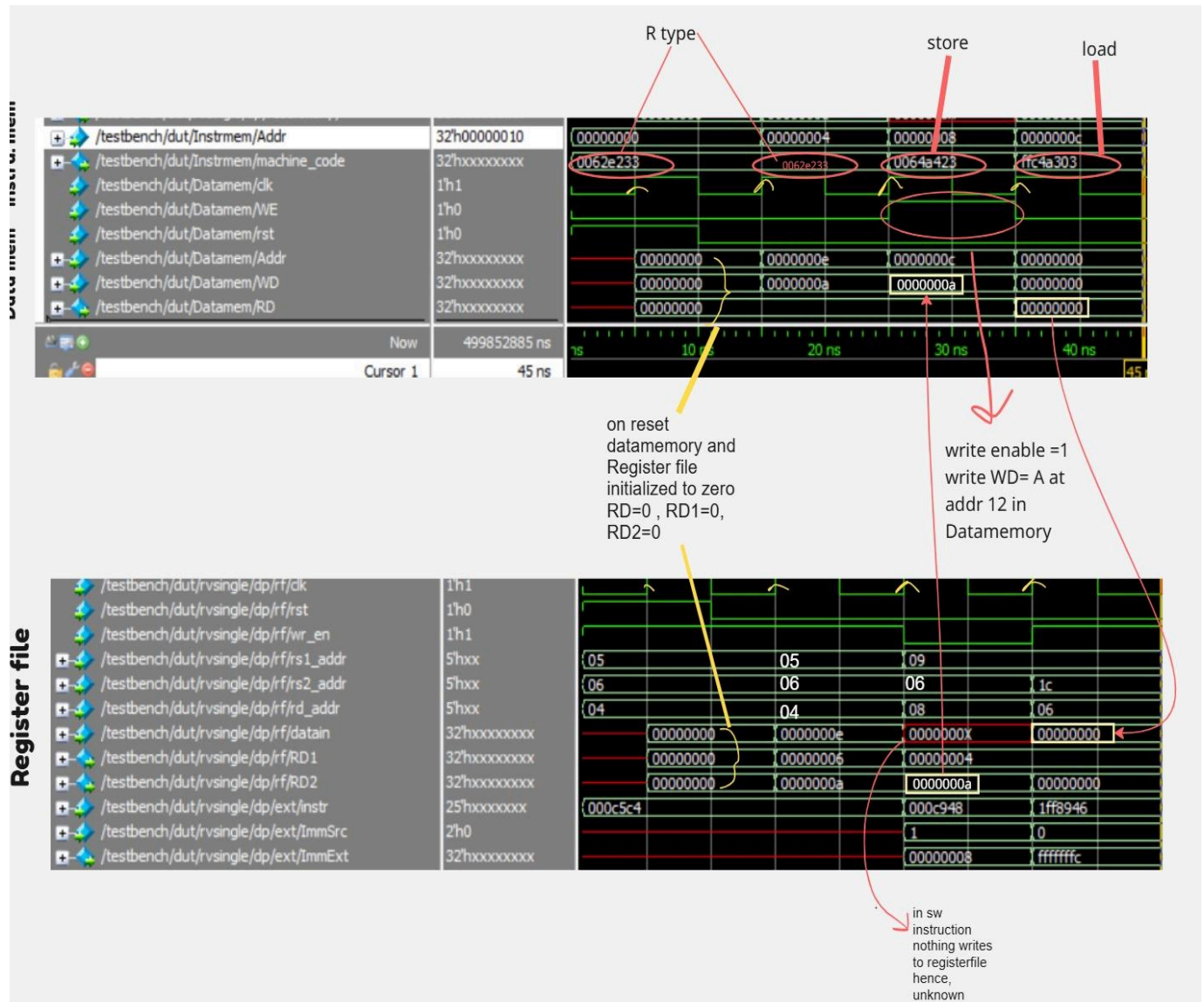
[illegible]

Memory Data - /testbench/dut/instmem/RAM - Default							
0000003f	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX
00000038	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX
00000031	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX
0000002a	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX
00000023	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX
0000001c	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX
00000015	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX
0000000e	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX
00000007	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	ffc4a303	0064a423	0062e233
00000000	0062e233						
ffffffff							

```

Memory Data - /testbench/dut/Datamem/RAM - Default
00000041 00000000 00000000 00000000 00000000 00000000 00000000 00000000
0000003a 00000000 00000000 00000000 00000000 00000000 00000000 00000000
00000033 00000000 00000000 00000000 00000000 00000000 00000000 00000000
0000002c 00000000 00000000 00000000 00000000 00000000 00000000 00000000
00000025 00000000 00000000 00000000 00000000 00000000 00000000 00000000
0000001e 00000000 00000000 00000000 00000000 00000000 00000000 00000000
00000017 00000000 00000000 00000000 00000000 00000000 00000000 00000000
00000010 00000000 00000000 00000000 00000000 00000000 00000000 00000000
00000009 00000000 00000000 00000000
00000002

```

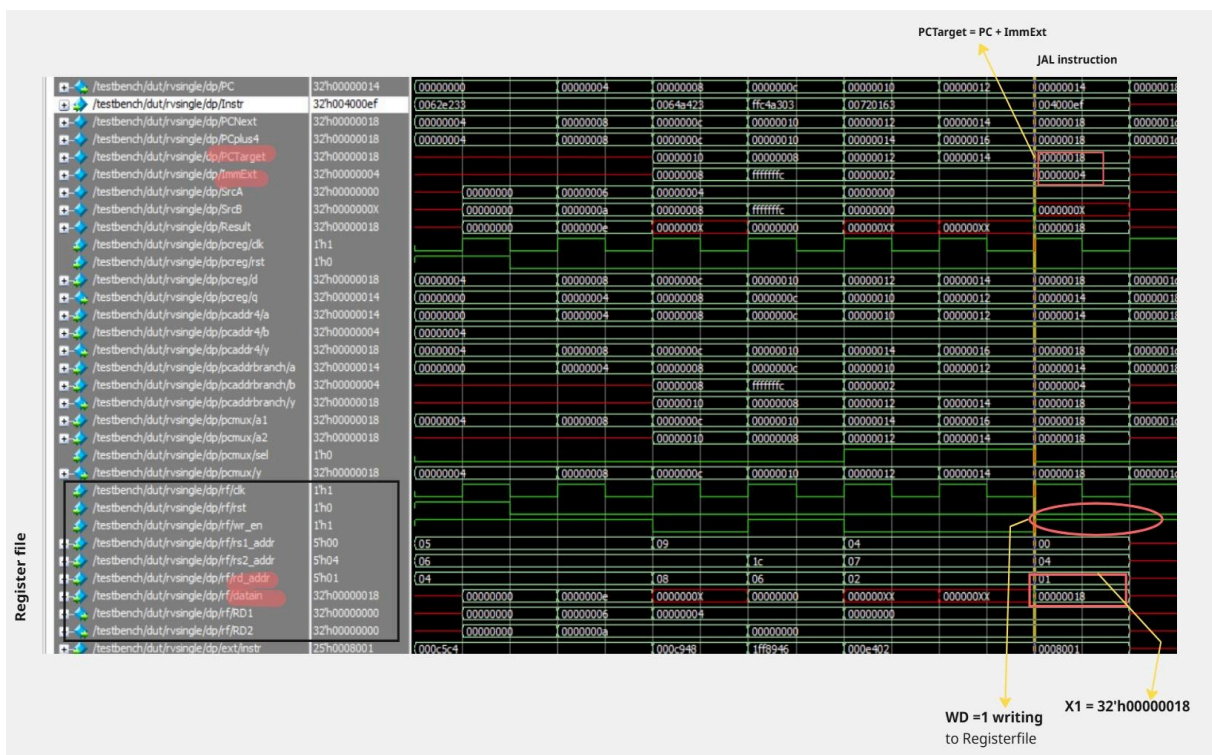
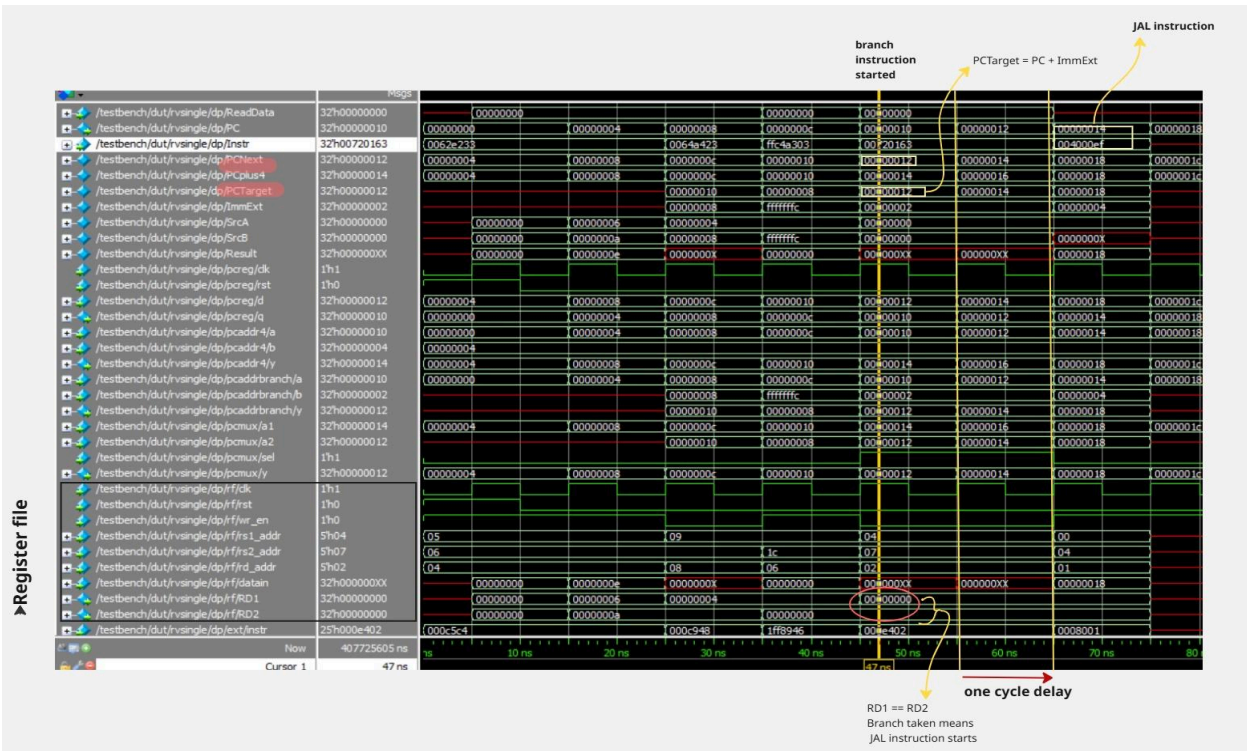


3) After successful verifying those 3 instructions , adding beq and jal instruction

B-type beq x4, x7, +4 if x4=x7 then Branch taken (PCTarget = PC + immext)
offset = +4 (immediate value) meaning next Jal instruction is going to execute

J-type jal x1, +4 forwards jump by +4 (PCTarget = PC + 4) and writing this obtained value in X1 register

Results If Branch Taken :



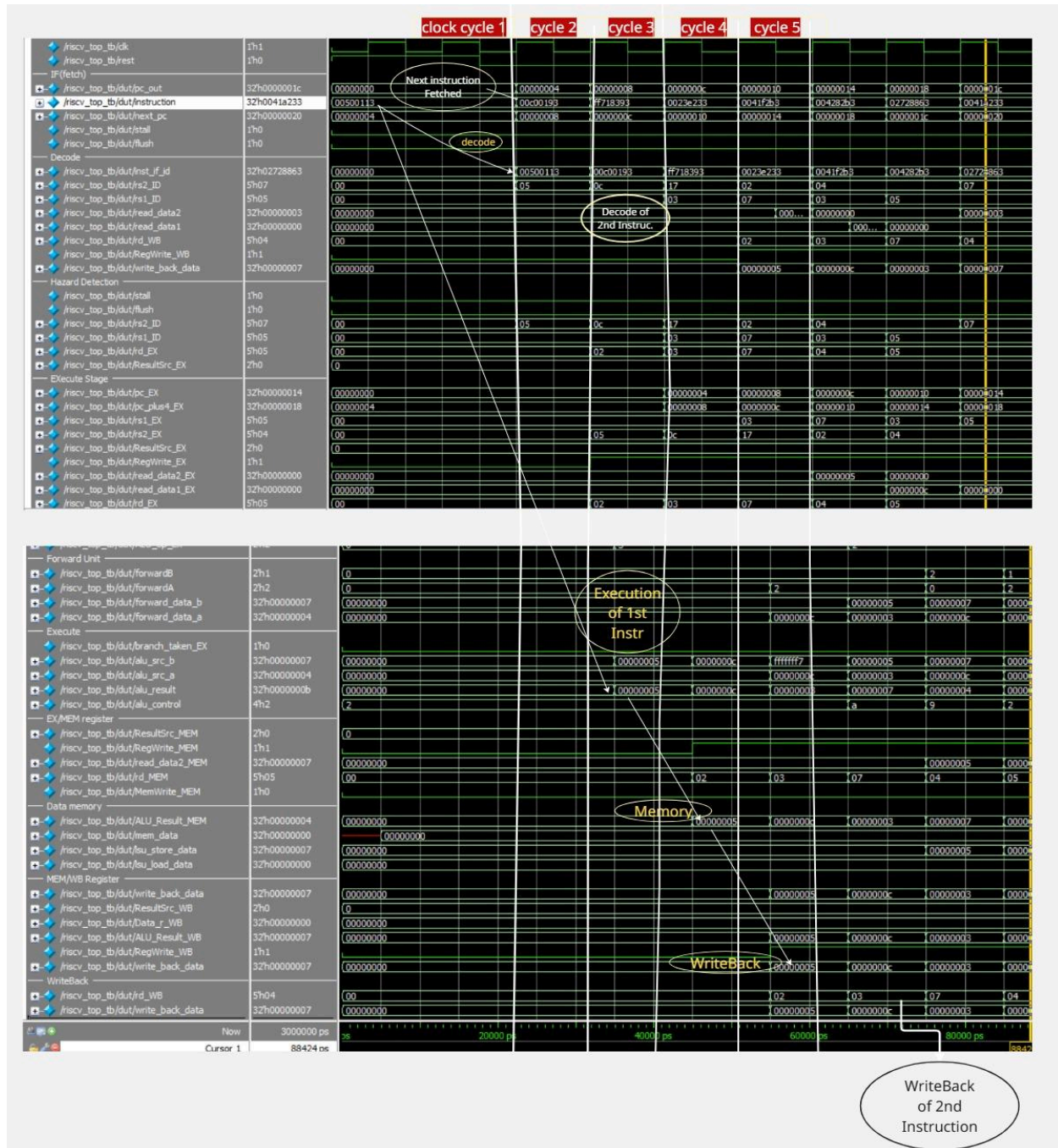
4) machine codes are not fetched when reset is deasserted (rst=0)

Root Cause:

Check PC , PCNext values in waveform and then correct the verilog logic.

5 Stage Pipeline RISCv Debugging

Simulated set of instructions inspected Pipeline Behavior.



Verifying Data Hazards dependency:

We can observe that 3rd instruction reading x3 register first before it get updated(writeback) by 2nd instruction, this is RAW Dependency. Therefore , forwarding unit helps in directly passing the content at x3 register from EX Stage only.

